



UNIDAD III. Estándares para el desarrollo web	ACTIVIDAD: DSW-A10. Página HTML, CSS, JavaScript, AJAX – Centro de Cómputo FEI
TIPO: Actividad individual	

OBJETIVO:

El estudiante incorporará carga dinámica de información a una página web institucional previamente desarrollada con HTML, CSS y JavaScript, mediante el uso de **JSON**, **AJAX** y la función **fetch()**, para obtener datos de archivos locales, procesarlos como objetos JavaScript y mostrarlos en el documento HTML sin recargar la página. Además, aplicará el manejo de promesas con `.then()` y `.catch()`, así como el uso de funciones flecha para actualizar contenido del portal de manera asíncrona.

DESCRIPCIÓN:

El estudiante retomará la página web del Portal Institucional del Centro de Cómputo FEI, trabajada en la actividad anterior DSW-A09. Página HTML, CSS y JavaScript – Centro de Cómputo FEI, en la cual ya se incorporaron interactividad, manipulación del DOM, eventos, validación de formulario, drag and drop y modo nocturno

A partir de esa base, ahora integrará carga dinámica de datos usando archivos JSON y la técnica AJAX mediante la función `fetch()`, tal como se explica en las diapositivas de clase UnidadIII_Estandares_para_el_desarrollo_webP3v4.pdf.

Durante la práctica, creará archivos JSON con información institucional del portal, realizará peticiones asíncronas para obtener esos datos y actualizará distintas secciones de la página sin necesidad de recargarla. También mostrará mensajes de carga o error en el panel dinámico, construirá contenido HTML a partir de arreglos de objetos y aplicará una petición simulada para convertir la información del formulario en formato JSON.

La finalidad es que el estudiante comprenda que AJAX permite solicitar y procesar información en segundo plano, mientras que JSON proporciona una forma ligera y estructurada de intercambiar datos entre aplicaciones. Asimismo, se busca que identifique a `fetch()` como una alternativa moderna a `XMLHttpRequest`, basada en promesas y orientada a un desarrollo web más claro y mantenible.

MATERIAL Y EQUIPO REQUERIDOS:

- a) Visual Studio Code o cualquier otro editor de código.
- b) Navegador web actualizado (Chrome o Firefox).
- c) Carpeta del proyecto desarrollada en la actividad DSW-A09.
- d) Archivo `index.html`.
- e) Archivo `css/estilos.css`.
- f) Archivo `js/script.js`.
- g) Crear una carpeta llamada *datos* para almacenar los archivos JSON.
- h) Imágenes del portal en la carpeta `img`, en caso de haberlas usado previamente.



PROCEDIMIENTO:

Paso 1 – Preparar la carpeta de datos JSON

Realiza una copia de la carpeta del proyecto creado en la actividad DSW-A09. Dentro de la carpeta principal del proyecto, crea una nueva carpeta llamada:

`datos`

En esa carpeta se almacenarán los archivos JSON que serán cargados dinámicamente desde la página web mediante `fetch()`.

Paso 2 – Verificar la referencia al archivo JavaScript

Abre el archivo `index.html` y comprueba que, antes de cerrar la etiqueta `</head>`, se encuentre la referencia al archivo JavaScript:

```
<script src="js/script.js" defer></script>
```

El atributo `defer` permite que el script se ejecute una vez que el documento HTML haya sido interpretado por el navegador, evitando errores al intentar acceder a elementos que todavía no existen en el DOM. Esta estructura ya fue utilizada en la práctica anterior.

Paso 3 – Crear el archivo `avisos.json`

En la carpeta `datos`, crear el archivo:

`avisos.json`

Y agregar el siguiente contenido:

```
[
  {
    "titulo": "Mantenimiento programado de red",
    "fecha": "10 de junio de 2026",
    "descripcion": "Se realizarán tareas de mantenimiento en la red institucional de 7:00 a 9:00 horas."
  },
  {
    "titulo": "Actualización de software en laboratorios",
    "fecha": "12 de junio de 2026",
    "descripcion": "Se instalarán nuevas versiones de software académico en los equipos del laboratorio."
  },
  {
    "titulo": "Convocatoria para préstamo de equipo",
    "fecha": "14 de junio de 2026",
    "descripcion": "Se habilita el registro para solicitar préstamo temporal de equipo de cómputo."
  }
]
```



Este archivo contiene un arreglo de objetos JSON, donde cada objeto representa un aviso institucional con sus propiedades título, fecha y descripción.

Paso 4 – Crear el archivo indicadores.json

En la misma carpeta `datos`, crear el archivo:

`indicadores.json`

Con el siguiente contenido:

```
[
  {
    "nombre": "Incidentes atendidos",
    "valor": 128
  },
  {
    "nombre": "Equipos disponibles",
    "valor": 42
  },
  {
    "nombre": "Solicitudes resueltas",
    "valor": 97
  }
]
```

Este segundo archivo será utilizado más adelante para mostrar datos en otra sección del portal.

Paso 5 – Agregar un contenedor para los avisos dinámicos

En `index.html`, localiza la sección de **Avisos y comunicados** y debajo del encabezado correspondiente agrega el siguiente contenedor vacío:

```
<div id="lista-avisos"></div>
```

Elimina los avisos existentes:

```
<!-- Artículo 1 -->
    <article class="aviso" aria-labelledby="aviso-1-titulo">
...
    </article>

<!-- Artículo 2 -->
    <article class="aviso" aria-labelledby="aviso-2-titulo">
...
    </article>
```



La estructura debe quedar de la siguiente forma:

```
<section aria-label="Novedades y avisos">
  <h2 id="avisos">Avisos y comunicados</h2>
  <div id="lista-avisos"></div>
</section>
```

Este contenedor servirá para insertar dinámicamente los avisos cargados desde el archivo avisos.json.

Paso 6 – Crear un contenedor para indicadores dinámicos

Ahora localiza la sección de **Indicadores** en `index.html` y agrega, debajo del encabezado principal, el siguiente bloque:

```
<div id="panel-indicadores"></div>
```

Elimina los indicadores existentes:

```
<section aria-label="Indicadores operativos">
...
</section>

<section aria-label="Indicadores de infraestructura">
...
</section>
```

El fragmento de código debe quedar de la siguiente forma:

```
<!-- INDICADORES (LISTAS) -->
<section id="indicadores" aria-labelledby="indicadores-titulo">
  <h2 id="indicadores-titulo">Indicadores</h2>
  <div id="panel-indicadores"></div>
</section>
```

Más adelante, este contenedor se llenará con información obtenida desde `indicadores.json`.

Paso 7 – Mostrar un mensaje inicial en el panel dinámico

En la práctica DSW-A09 se creó un panel de mensajes dinámicos con el párrafo `#mensaje-dinamico` para informar al usuario sobre el estado del portal. Ese mismo panel se utilizará ahora para notificar la carga de datos con AJAX.

Abre `js/script.js` y sustituye el código:

```
const mensajeDinamico = document.getElementById("mensaje-dinamico");
```



```
mensajeDinamico.textContent = "Portal cargado correctamente. JavaScript se  
encuentra activo.";
```

Por el siguiente:

```
const mensajeDinamico = document.getElementById("mensaje-dinamico");  
mensajeDinamico.textContent = "Cargando información del portal...";
```

Este mensaje indicará que el portal está iniciando la carga de datos desde archivos JSON.

Paso 8 – Cargar indicadores con fetch() y construir tarjetas

Al final del archivo `script.js`, agrega el siguiente código:

```
//Carga de indicadores con fetch()  
  
fetch("datos/indicadores.json")  
.then(respuesta => respuesta.json())  
.then(indicadores => {  
    const panelIndicadores = document.getElementById("panel-indicadores");  
  
    indicadores.forEach(indicador => {  
        const bloque = document.createElement("article");  
        bloque.classList.add("tarjeta-indicador");  
  
        bloque.innerHTML = `  
            <h3>${indicador.nombre}</h3>  
            <p>${indicador.valor}</p>  
        `;  
  
        panelIndicadores.appendChild(bloque);  
    });  
})  
.catch(error => {  
    console.log("Error al cargar indicadores:", error);  
});
```



Indicadores

Incidentes atendidos

128

Equipos disponibles

42

Solicitudes resueltas

97

Paso 9 – Agregar estilos para los indicadores

En css/estilos.css, agregar las siguientes reglas:

```
/*Estilos indicadores*/
#panel-indicadores {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 12px;
  margin-top: 10px;
}

.tarjeta-indicador {
  background-color: #f8fbff;
  border: 1px solid #18529D;
  border-radius: 6px;
  padding: 14px;
  text-align: center;
}

.tarjeta-indicador h3 {
  margin-top: 0;
  color: #18529D;
}

.tarjeta-indicador p {
  font-size: 1.4rem;
  font-weight: bold;
  color: #28AD56;
}
```

En tu página deben verse de la siguiente forma, después de aplicar el estilo:



Indicadores

Incidentes atendidos

128

Equipos disponibles

42

Solicitudes resueltas

97

Paso 10- Crear una función para cargar los avisos desde avisos.json y agregar botón “Restaurar avisos”

Se creará la función `cargarAvisos` y será llamada tanto al inicio del script como al hacer clic en un botón llamado “Cargar avisos” ubicado en la barra de acciones rápidas.

En `script.js` ubica este bloque:

```
accionesRapidas.innerHTML = `  
  <h2>Acciones rápidas</h2>  
  <p>Utiliza los siguientes botones para interactuar con el portal.</p>  
  <button type="button" id="btn-resaltar-avisos">Resaltar avisos</button>  
  <button type="button" id="btn-ocultar-primer-aviso">Eliminar primer  
aviso</button>  
  <button type="button" id="btn-restaurar-mensaje">Restaurar  
mensaje</button>  
`;  
`;
```

Modifícalo así:

```
accionesRapidas.innerHTML = `  
  <h2>Acciones rápidas</h2>  
  <p>Utiliza los siguientes botones para interactuar con el portal.</p>  
  <button type="button" id="btn-resaltar-avisos">Resaltar avisos</button>  
  <button type="button" id="btn-ocultar-primer-aviso">Eliminar primer  
aviso</button>  
  <button type="button" id="btn-cargar-avisos">Cargar avisos</button>  
  <button type="button" id="btn-restaurar-mensaje">Restaurar  
mensaje</button>  
`;  
`;
```

Se creará la función `cargarAvisos` para utilizarla al cargar la página y en el botón “Cargar Avisos”. Dentro de ella se empleará `fetch()`. Al inicio de `script.js` agrega:

```
//Función reutilizable para cargar avisos
```

```
const cargarAvisos = () => {
```



```
const mensajeDinamico2 = document.getElementById("mensaje-dinamico");
mensajeDinamico2.textContent = "Cargando avisos del portal...";

fetch("datos/avisos.json")
  .then(respuesta => respuesta.json())
  .then(avisos => {
    const listaAvisos = document.getElementById("lista-avisos");

    //Limpiar contenido previo (IMPORTANTE)
    listaAvisos.innerHTML = "";

    avisos.forEach(avisos => {
      const tarjetaAviso = document.createElement("article");
      tarjetaAviso.classList.add("aviso");

      tarjetaAviso.innerHTML = `
        <h3>${avisos.titulo}</h3>
        <p><strong>Fecha:</strong> ${avisos.fecha}</p>
        <p>${avisos.descripcion}</p>
      `;

      listaAvisos.appendChild(tarjetaAviso);
    });

    mensajeDinamico2.textContent = `Se cargaron ${avisos.length} avisos
correctamente.`;
  })
  .catch(error => {
    mensajeDinamico2.textContent = "Error al cargar los avisos del
portal.";
    console.log("Ocurrió un error:", error);
  });
};
```

*Nota importante: Debes agregar la función al inicio de script.js, ya que las funciones flecha no se pueden llamar antes de su declaración, es decir las Arrow functions no tienen hoisting.

Después de declarar la función agrega la llamada:

```
//Llamada a la función cargarAvisos, para que se carguen desde el inicio
cargarAvisos();
```

Dentro del mismo **script.js** ubica los eventos de “Acciones rápidas” y al final de ellos agrega el evento para el botón “Cargar avisos”:

```
document.getElementById("btn-cargar-avisos").addEventListener("click",
cargarAvisos);
```



De esta forma los avisos se cargarán cuando abras la página y cuando hagas clic en el botón “Cargar avisos”:

Avisos y comunicados

Mantenimiento programado de red
Fecha: 10 de junio de 2026
Se realizarán tareas de mantenimiento en la red institucional de 7:00 a 9:00 horas.

Actualización de software en laboratorios
Fecha: 12 de junio de 2026
Se instalarán nuevas versiones de software académico en los equipos del laboratorio.

Convocatoria para préstamo de equipo
Fecha: 14 de junio de 2026
Se habilita el registro para solicitar préstamo temporal de equipo de cómputo.

Acciones rápidas

Utiliza los siguientes botones para interactuar con el portal.

Resaltar avisos

Eliminar primer aviso

Cargar avisos

Restaurar mensaje

Hasta el momento tu portal web debe verse de la siguiente manera:



UNIVERSIDAD VERACRUZANA. Licenciatura en Ingeniería de Ciberseguridad e Infraestructura de Cómputo

EE. Desarrollo de Sistemas Web



Universidad Veracruzana
Facultad de Estadística e Informática

[Inicio](#) [Servicios](#) [Horarios](#) [Indicadores](#) [Contacto](#)

[Activar modo nocturno](#)

Estado del portal

Se cargaron 3 avisos correctamente.

Portal Institucional del Centro de Cómputo

Información oficial sobre servicios, avisos, horarios de atención y recursos para la comunidad. Esta página está construida con HTML5, servidoro para facilitar accesibilidad, mantenimiento y SEO.

Accesos rápidos: [Ver servicios](#) | [Reportar incidente](#)

Galería del Centro de Cómputo



Vista general del laboratorio.

[Anterior](#) [Siguiente](#)

Avisos y comunicados

Mantenimiento programado de red

Fecha: 10 de junio de 2020

Se realizarán tareas de mantenimiento en la red institucional de 7:00 a 9:00 horas.

Actualización de software en laboratorios

Fecha: 10 de junio de 2020

Se instalarán nuevas versiones de software académico en los equipos del laboratorio.

Convocatoria para préstamo de equipo

Fecha: 14 de junio de 2020

Se habilita el registro para solicitar préstamo temporal de equipo de cómputo.

Acciones rápidas

Utiliza los siguientes botones para interactuar con el portal.

[Resaltar avisos](#) [Eliminar primer aviso](#) [Cargar avisos](#) [Restaurar mensajes](#)

Servicios

Servicios disponibles para estudiantes, docentes y personal administrativo.

Mesa de ayuda

Atención de incidentes: conectividad, cuentas, software, periféricos y aulas.

Préstamo y resguardo de equipo

Solicitud, control y seguimiento de préstamos temporales de equipo.

Acceso a laboratorios

Políticas de uso, horarios, insumos y buenas prácticas de seguridad.

Horarios de atención

Horarios referenciales. Confirma cambios en avisos.

Horario de atención por servicio

Servicio	Días	Horario	Ubicación
Mesa de ayuda	Lunes a viernes	09:00-18:00	Edificio A - Planta baja
Laboratorios	Lunes a sábado	07:00-20:00	Edificio B - Aulas de cómputo
Préstamo de equipo	Lunes a viernes	10:00-14:00	Centro de Cómputo

Indicadores

Incidentes atendidos

128

Equipos disponibles

42

Solicitudes resueltas

97

Recursos

Enlaces útiles

- [Especialidad de Ingeniería](#)
- [Carreras académicas](#)
- [Directorio](#)

Accesibilidad

Este portal busca seguir buenas prácticas de accesibilidad: estructura semántica, textos alternativos y formularios con etiquetas claras.

Paso 11 – Convertir información del formulario a JSON

A continuación, se obtendrán los datos del formulario de contacto para construir un objeto JavaScript y convertirlo a JSON con `JSON.stringify()`.



En `index.html` asigna un identificador al formulario:

```
<form id="formulario-contacto" action="#" method="post" aria-label="Formulario de contacto" novalidate>
```

Agrega en `script.js` el siguiente bloque:

```
//Convertir datos del formulario a JSON

document.getElementById("formulario-contacto").addEventListener("submit",
function (event) {
    event.preventDefault();

    const solicitud = {
        nombre: document.getElementById("nombre").value,
        correo: document.getElementById("correo").value,
        rol: document.getElementById("rol").value,
        tipo: document.getElementById("tipo").value,
        descripcion: document.getElementById("descripcion").value
    };

    /*Convierte el objeto javascript "solicitud" en una cadena JSON
    estableciendo el parámetro replacer=null y el parámetro space=2*/
    const solicitudJSON = JSON.stringify(solicitud, null, 2);

    document.getElementById("salida-json").textContent = solicitudJSON;
});
```

Y debajo del formulario, en `index.html`, agrega el contenedor donde se mostrará el JSON generado:

```
<section id="json-formulario" aria-labelledby="json-formulario-titulo">
<h3 id="json-formulario-titulo">Datos del formulario en formato JSON</h3>
<pre id="salida-json"></pre>
</section>
```

Paso 12 – Agregar estilos para la salida JSON

```
/*Datos del form en JSON*/
#salida-json {
    background-color: #f4f6f9;
    border: 1px solid #d0d7de;
    border-radius: 6px;
    padding: 12px;
    overflow-x: auto;
    white-space: pre-wrap;
```



```
margin-top: 10px;  
}
```

Datos del formulario en formato JSON

```
{  
  "nombre": "Erika Meneses Rico",  
  "correo": "ermenteses@uv.mx",  
  "rol": "docente",  
  "tipo": "red",  
  "descripcion": "Texto descriptivo del incidente"  
}
```

Paso 12 – Verificar el funcionamiento general de la página

Comprobar la funcionalidad completa:

- a) El panel dinámico muestra el mensaje de carga inicial.
- b) Los avisos se cargan desde avisos.json al cargar inicialmente la página y se insertan dinámicamente en la sección correspondiente.
- e) El botón **Cargar avisos** vuelve a generar la información sin recargar la página.
- d) Los indicadores se cargan desde indicadores.json y aparecen en formato de tarjetas.
- f) El formulario de contacto convierte los datos capturados a formato JSON y los muestra en pantalla.
- h) El resto de funcionalidades desarrolladas en la práctica anterior continúan operando correctamente.

RESULTADOS ENTREGABLES

1. Un archivo .zip con la carpeta completa del proyecto.
2. El proyecto debe incluir correctamente organizados los siguientes archivos:
 - index.html
 - css/estilos.css
 - js/script.js
 - carpeta datos
 - carpeta img
3. La página debe funcionar correctamente en navegador.
4. El código debe conservar una estructura ordenada, indentación adecuada y comentarios en las secciones principales.



5. Entregar capturas de pantalla que evidencien:
- La carga dinámica de avisos,
 - La carga dinámica de indicadores,
 - El total de avisos mostrados,
 - La salida del formulario en formato json,
 - El panel de mensajes dinámicos actualizado.